Skills Field Manual

10 Claude Capabilities That Replace
Generic Prompting

What each one does. When to use it.
When to skip it. How to set it up right.

Clief Notes | Premium Member Resource

v1.0 | March 2026

Created by Jake Van Clief | Eduba

This document is for Clief Notes Premium and VIP members only.
Redistribution without permission is prohibited.

Contents

1 Read This First **2**

2 Context Architecture Skills **2**
2.1 Skill 1: Projects . . . 2
2.1.1 The approach . . . 3
2.2 Skill 2: Custom Skills . . . 3
2.2.1 The approach . . . 4
2.3 Skill 3: Claude Code + CLAUDE.md . . . 4
2.3.1 The approach . . . 5
2.4 Skill 4: MCP Connectors . . . 5
2.4.1 The approach . . . 6
2.5 Skill 5: Memory . . . 6
2.5.1 The approach . . . 7

3 Output Mode Skills **7**
3.1 Skill 6: Code Execution . . . 7
3.1.1 The approach . . . 7
3.2 Skill 7: Artifacts . . . 8
3.2.1 The approach . . . 8
3.3 Skill 8: File Creation . . . 8
3.3.1 The approach . . . 9
3.4 Skill 9: Web Search + Deep Research . . . 9
3.4.1 The approach . . . 9
3.5 Skill 10: Extended Thinking . . . 10
3.5.1 The approach . . . 10

4 Putting It Together **10**
4.1 The decision sequence . . . 11

5 Quick Reference **11**

1 Read This First

A prompt is a string of text you type into a chat window. A skill is a capability built into the tool. The difference matters because most AI education teaches people to write better strings when the real leverage is in choosing the right capability for the job.

This manual covers ten Claude capabilities that, used correctly, replace dozens of generic prompt templates. Each one is a tool. Each tool has a right time and a wrong time. The guide for each skill follows the same format:

- **What it does.** One paragraph. No hype.
- **Use it when.** The situations where this skill saves you real time.
- **Skip it when.** The situations where this skill adds friction or cost.
- **The approach.** How to set it up so it actually works.
- **60/30/10 layer.** Where this skill sits in the framework.

The core principle. The biggest lever in AI output quality is not the prompt. It is what information the model can see when it processes that prompt. Context architecture beats prompt wordsmithing every time. Five of the ten skills in this guide are about controlling what the model sees. The other five are about choosing the right output mode.

2 Context Architecture Skills

These five skills control what information Claude has access to when it thinks about your problem. They are the highest-leverage tools in the stack.

2.1 Skill 1: Projects

What it does. Projects are persistent workspaces inside Claude. Each project has its own knowledge base (up to 200K tokens, roughly 500 pages), custom instructions, and chat history. Everything you upload to a project's knowledge base is available in every conversation within that project. Custom instructions act as a system prompt that shapes every response.

Use it when:

- You return to the same body of work repeatedly (a client engagement, a product, a research area).
- You need Claude to know your company's terminology, style guide, or domain context without re-explaining it.
- A team needs shared access to the same AI workspace with the same knowledge base.
- You find yourself uploading the same documents to new conversations.

Skip it when:

- The task is a one-off question that needs no persistent context.
- You are doing quick research or brainstorming that does not reference specific documents.
- Your knowledge base exceeds the 200K token limit (you will need to be selective about what goes in).

2.1.1 The approach

The custom instructions field is the most underused part of Projects. Most people leave it blank or put something vague like "You are a helpful assistant for my company." That wastes the most powerful slot in the entire system.

Write your custom instructions like an onboarding document for a new hire. Include: who you are, what the project is about, what terminology means in your context, what the output format should look like, and what Claude should refuse to do or flag for human review.

Example: Custom instructions for a client engagement project

```
You are assisting with the Pacific Life AI training engagement.
Context:  - Client:  Pacific Life, insurance and retirement company - Engagement:
AI literacy training for 300+ employees - Delivery partner:  Correlation One - Our
role:  curriculum design and lead instruction
Key terminology:  - "Orchestration" = choosing the right layer (AI, code, human) for
each task - "60/30/10" = our framework:  60% database, 30% rules, 10% AI - "Learn
First, Build Right" = our methodology
When I ask you to draft something for the client:  - Use professional but plain
language, no jargon they haven't been taught - Never promise capabilities we haven't
demonstrated - Flag anything that touches compliance or regulated processes
When I ask you to build training materials:  - Follow the session structure in the
uploaded curriculum doc - Include hands-on exercises, not just slides - Reference
real Pacific Life workflows where possible
```

60/30/10 layer: Projects are infrastructure. They support work at every layer. The knowledge base can hold database schemas (60%), decision tree documentation (30%), and prompt templates (10%). The custom instructions determine which layer Claude defaults to.

2.2 Skill 2: Custom Skills

What it does. Skills are folders containing instructions, scripts, and reference files that Claude loads on demand when they are relevant to your task. They work in both Claude.ai (via Settings > Customize > Skills) and Claude Code (via `.claude/skills/` directories). Each skill has a `SKILL.md` file with a name, description, and instructions. Claude reads the description to decide when to activate a skill automatically.

The difference between Skills and Projects: Projects give Claude persistent context about a body of work. Skills teach Claude how to do something. A Project says "here is everything about this client." A Skill says "here is how to build a PowerPoint deck that follows our brand guidelines."

Use it when:

- You find yourself typing the same instructions across multiple conversations or projects.
- You need consistent output formatting (brand guidelines, document templates, code standards).
- You want Claude to run a specific workflow with steps, scripts, or validation logic.
- You have a repeatable process that should produce the same quality every time.

Skip it when:

- The task only happens once.
- The instructions are simple enough to fit in a single prompt.
- You are still figuring out the right process (get it working manually first, then make it a skill).

2.2.1 The approach

Start by noticing what you repeat. If you copy-paste the same instructions into three different conversations, that is a skill waiting to be packaged. The structure is straightforward:

Skill folder structure

```
my-skill/ SKILL.md # Instructions (required) REFERENCE.md # Supplemental info Claude
reads if needed templates/ # Output templates examples/ # Good input/output pairs
scripts/ # Code Claude can run
```

The `SKILL.md` file starts with YAML frontmatter (name and description) followed by markdown instructions. The description field is critical. Claude uses it to decide when to activate the skill. Write it like a trigger condition: "Apply Acme brand guidelines to presentations and documents, including official colors, fonts, and logo usage."

Built-in skills already exist for creating Word documents, PowerPoint decks, spreadsheets, and PDFs. You do not need to build these yourself. But you can build skills that wrap the built-in ones with your own standards.

60/30/10 layer: Skills operate at the 30% layer most naturally. They encode rules, templates, and repeatable processes. The skill itself is rule-based logic. Claude's role is filling in the blanks with judgment where the template requires it.

2.3 Skill 3: Claude Code + CLAUDE.md

What it does. Claude Code is a command-line agent that reads your project files, writes code, runs commands, and navigates your codebase. The `CLAUDE.md` file in your project root is auto-loaded into every conversation. It acts as a persistent system prompt that tells Claude Code how your project is organized, what conventions to follow, and where to find things.

In Claude Code, subagents are markdown files in `.claude/agents/` with YAML frontmatter. Each subagent has its own name, description, and system prompt. You can give subagents specific tool permissions and scope. They can be invoked with slash commands or triggered automatically.

Use it when:

- You are building or maintaining software and want an AI agent that understands your full codebase.
- You need multi-step task execution: research, plan, implement, test.
- You want to delegate specific types of work to specialized subagents (testing, documentation, security review).
- Your project has enough structure that a `CLAUDE.md` file can orient the agent.

Skip it when:

- You are not working in a codebase. For writing, analysis, or research, use Claude.ai with Projects.
- The task is small enough to handle in a single chat message.
- You do not have a well-organized project structure (fix that first).

2.3.1 The approach

The `CLAUDE.md` file is the single highest-leverage artifact in a Claude Code project. Write it the way you would write orientation notes for a contractor starting their first day. Include: a folder map, naming conventions, key dependencies, how to run tests, and a routing table that says "doing X? look in Y directory."

Example: CLAUDE.md routing table

```
## Task Routing
| Task type | Start here | |------------|------------------| | New API endpoint
| src/api/ + read CONVENTIONS.md | | Frontend component | src/components/ + read
STYLE.md | | Database migration | db/migrations/ + read SCHEMA.md | | Bug fix | Run
tests first, then investigate | | Documentation | docs/ + match existing format |
## Conventions - All API responses use the envelope format in src/api/types.ts -
Tests live next to the code they test (*.test.ts) - Commit messages:  type(scope):
description
```

Subagents are most useful when different tasks need different thinking. A test-writer agent should be strict and thorough. A documentation agent should be clear and concise. Giving each one a focused system prompt produces better results than asking one general agent to switch modes.

60/30/10 layer: Claude Code operates across all three layers. The `CLAUDE.md` file and subagent definitions are 30% (rules and routing). The code Claude writes might be 60% (deterministic scripts) or 10% (novel problem-solving). Your job is to make the routing clear enough that Claude picks the right approach.

2.4 Skill 4: MCP Connectors

What it does. Model Context Protocol (MCP) is an open standard that connects Claude to external tools: Google Drive, Gmail, Slack, Jira, GitHub, Notion, Stripe, and thousands more through Zapier. When connected, Claude can read your actual data, create tasks, send messages, and take actions in the tools you already use. Claude.ai has a growing list of built-in connectors. Claude Code supports any MCP server.

Use it when:
- You need Claude to work with your real data, not data you copy-paste into the chat.
- You want Claude to take actions (create a calendar event, file a Jira ticket, send an email).
- Your workflow crosses multiple tools and you are tired of being the glue between them.
- You are building agentic workflows where Claude needs to read from and write to external systems.

Skip it when:
- The data is small enough to paste directly into the conversation.
- You are working with sensitive data and have not reviewed the connector's access permissions.
- The external tool has an API rate limit that will bottleneck your workflow.
- You just need Claude to think, not to act. Read-only analysis of pasted data does not need MCP.

2.4.1 The approach

Enable connectors in Claude.ai through the tools menu (look for the plug icon). Each connector asks for authentication. Start with read-only access until you trust the workflow, then enable write access.

In Claude Code, MCP servers are configured in your project settings. You can also build custom MCP servers for internal tools using the Anthropic SDK (Python or Node.js). A custom MCP server wraps your internal API in a format Claude can understand and call.

The power of MCP is combining it with Projects. A project for "Q4 Sales Analysis" with Google Drive and Slack connected lets Claude pull actual sales data from your Drive and post summaries to your Slack channel without you acting as the middleman.

60/30/10 layer: MCP connectors are plumbing. They connect layers. The action Claude takes through a connector might be 60% (pull a report), 30% (route a ticket based on rules), or 10% (draft a response that requires judgment). The connector itself is infrastructure, not intelligence.

2.5 Skill 5: Memory

What it does. Memory lets Claude retain information about you across conversations. It builds over time from your interactions. You can also tell Claude to remember specific facts ("Remember that I work at Acme Corp") or forget them ("Forget about my old job"). Memory is personal to your account and applies outside of Projects.

Use it when:
- You use Claude as a general assistant across many topics and want it to know your context.
- You have preferences (writing style, technical level, formatting) that should apply everywhere.
- You are tired of re-introducing yourself and your work at the start of every conversation.

Skip it when:
- You are working inside a Project (Project context overrides general memory for that workspace).
- You need precise, structured context. Memory is best for lightweight personalization, not heavy reference material.
- You want to keep a conversation context-free (use incognito mode).

2.5.1 The approach

Memory works best as a background layer. Set your professional context (role, company, expertise level), communication preferences (tone, formatting, language), and any persistent facts that matter across conversations.

Do not rely on memory for project-specific context. That belongs in Projects. Memory is for the things that are true about you regardless of what you are working on.

60/30/10 layer: Memory is infrastructure. It calibrates all three layers by helping Claude choose the right vocabulary, technical depth, and output format for you specifically.

3 Output Mode Skills

These five skills control how Claude delivers its work. Choosing the right output mode for the task avoids the most common mistake in AI usage: asking for text when you need a file, or asking for a file when you need an answer.

3.1 Skill 6: Code Execution

What it does. Claude can write and run Python code inside a sandboxed environment during your conversation. This means it can do math, process data, generate charts, manipulate files, and test logic with actual execution rather than guessing. Results are real. If the code fails, Claude sees the error and can fix it.

Use it when:
- You need precise calculations, data analysis, or statistical work.
- You want to process a CSV, Excel file, or dataset you have uploaded.
- You need Claude to verify its own work (run the code, check the output).
- You want a chart, graph, or visualization built from real data.

Skip it when:
- The question is conceptual or analytical ("what should our pricing strategy be?").
- You need a quick answer Claude can give from its training data.
- The computation is simple enough that a spreadsheet formula handles it.

3.1.1 The approach

The trigger phrase that matters: tell Claude *what you want to know*, not what code to write. "What's the average deal size by quarter from this spreadsheet?" will produce better results than "Write a pandas script to calculate averages." Claude knows how to write the code. Let it choose the approach.

For complex data work, upload your file and describe the analysis in plain language. Claude will write the code, run it, show you the results, and iterate if something looks wrong. This is a genuine 60% task (data processing) that Claude handles well because it is running deterministic code, not guessing.

60/30/10 layer: Code execution is a 60% tool. The code is deterministic. The AI writes it, but the output is as reliable as any script. This is one of the best uses of AI: let the model write the code, then let the code do the work.

3.2 Skill 7: Artifacts

What it does. Artifacts are interactive, renderable outputs that appear alongside the conversation: React components, HTML pages, SVG graphics, Mermaid diagrams, markdown documents, and more. They can be shared, forked, and published. Artifacts with persistent storage can save data across sessions.

Use it when:
- You need an interactive tool (calculator, dashboard, form, decision tree).
- You want a visual output (diagram, flowchart, data visualization).
- You are building a prototype or proof of concept to share with others.
- You need a document that will be edited, expanded, or iterated on.

Skip it when:
- The answer fits in a few sentences of conversation.
- You need a production-quality file format (use file creation for .docx, .pptx, .xlsx).
- The content is a short list, a quick explanation, or a simple answer.

3.2.1 The approach

Describe the end state. "Build me an interactive ROI calculator where I input headcount, hourly rate, and hours saved per week, and it shows annual savings with a chart" gives Claude enough to build something useful in one pass.

Artifacts are best used as working tools, not finished deliverables. Build the interactive prototype in an artifact, test it, then export or rebuild it in your production stack.

60/30/10 layer: Artifacts themselves are 30% tools (interactive rule-based interfaces). The AI's role is writing the code that makes them work (10% creative generation), then the artifact runs deterministically.

3.3 Skill 8: File Creation

What it does. Claude can create professional documents in standard formats: Word (.docx), PowerPoint (.pptx), Excel (.xlsx), and PDF. These are real files with proper formatting, styles, charts, tables, and page layouts. Claude has built-in skills for each format that handle the technical details.

Use it when:
- You need a deliverable someone else will open in Office or Google Workspace.
- The output needs professional formatting: headers, tables of contents, page numbers, branded styles.
- You are producing a report, proposal, presentation, or spreadsheet for external use.
- You need to modify an existing file (edit a Word doc, update a slide deck, clean a spreadsheet).

Skip it when:
- The content is for your own reference (use markdown or artifacts).
- The document is short enough to copy from the chat.
- You need highly custom layouts that require manual design tool work.

3.3.1 The approach

Be specific about format, audience, and structure. "Create a 10-slide deck for our executive team on Q3 AI training results. Include a title slide, agenda, 3 data slides with charts, 4 content slides with key findings, a recommendations slide, and a next steps slide. Use the color scheme from the attached brand guide."

The more you define the structure up front, the less iteration you need. File creation works best when you treat Claude like a junior designer who is skilled at execution but needs a clear brief.

60/30/10 layer: File creation is a 30% task. The format and structure are rules. Claude's contribution is filling the template with content that requires judgment (10%). The file format handling is deterministic code running in the background (60%).

3.4 Skill 9: Web Search + Deep Research

What it does. Web search lets Claude pull current information from the internet during a conversation. It can verify facts, find recent news, check current prices or standings, and research topics that require up-to-date data. Deep Research is a more intensive mode that conducts multi-step research across many sources and produces a comprehensive report.

Use it when:
- You need current information (prices, news, who holds a position, recent events).
- You are researching a topic that changes frequently.
- You want Claude to verify a claim rather than relying on its training data.
- You need a comprehensive research report on a complex topic (use Deep Research).

Skip it when:
- The question involves established facts, definitions, or stable concepts.
- You are asking Claude to analyze data you have already provided.
- The topic is well within Claude's training data and has not changed recently.

3.4.1 The approach

For quick factual checks, just ask the question. Claude will search automatically when it determines the answer might have changed since its training cutoff.

For research tasks, be specific about what you need and what "good" looks like. "Research the competitive landscape for AI training companies targeting Fortune 500 enterprises. Focus on pricing models, delivery formats, and whether they do custom development alongside training. I need this for a positioning exercise, so emphasize where there are gaps in the market."

Deep Research handles the heavy lifting for topics that would otherwise require 20+ searches. If you are building a market analysis, competitive report, or literature review, point it at Deep Research and let it work.

60/30/10 layer: Web search is a lookup tool (60%). Deep Research is synthesis (10%). The retrieval is deterministic. The analysis and writing are where AI adds value.

3.5 Skill 10: Extended Thinking

What it does. Extended thinking lets Claude reason through complex problems step by step before producing its answer. It spends more time thinking internally, considering multiple angles, and checking its own logic. The thinking process is visible to you as a collapsible block above the response.

Use it when:
- The problem has multiple interacting constraints (system design, strategy, legal analysis).
- You need Claude to weigh trade-offs and explain its reasoning.
- The task involves math, logic, or technical analysis where precision matters.
- You want to verify Claude's thought process, not just its conclusion.

Skip it when:
- The question has a straightforward answer.
- Speed matters more than depth (quick edits, simple lookups, short responses).
- You are generating creative content where overthinking reduces quality.

3.5.1 The approach

Extended thinking activates automatically for complex tasks, but you can encourage it by framing problems that require multi-step reasoning: "Walk through the trade-offs of building this as a monolith versus microservices, considering our team size of 4, our 6-month runway, and the need to support 3 different client integrations."

The visible thinking block is useful for auditing. If Claude reaches the wrong conclusion, you can read the thinking to find where the logic went off track and correct it specifically.

60/30/10 layer: Extended thinking is a 10% tool. It exists specifically for the problems that require genuine judgment, synthesis, and multi-step reasoning. If the problem does not need this, you are paying for processing time you do not need.

4 Putting It Together

The ten skills above are not independent. They combine. A well-built workflow uses several of them together.

Example stack: Client engagement workflow

1. **Project** holds all client documents, meeting notes, and custom instructions defining your role and the engagement scope.
2. **MCP connectors** link Google Drive (shared client folder) and Slack (client channel) so Claude can access real documents and post updates.
3. **Custom Skill** defines how you produce client deliverables: your template structure, formatting standards, and review checklist.
4. **File creation** produces the actual .docx or .pptx deliverable using the skill's standards.
5. **Memory** knows your communication preferences, so outputs match your voice without you specifying it each time.

The key principle: each skill handles the thing it is best at. The project holds context. The connectors provide data. The skill defines process. The file creator handles output. The memory handles personalization. No single skill tries to do everything.

4.1 The decision sequence

When you sit down to do a task with Claude, run through these questions in order:

1. **Do I need persistent context?** Yes → use a Project. No → regular chat.
2. **Do I need external data?** Yes → enable MCP connectors or upload files. No → proceed.
3. **Is this a repeatable process?** Yes → build or use a Skill. No → prompt directly.
4. **What output format do I need?** File → file creation. Interactive → artifact. Data → code execution. Text → just talk.
5. **Does this need deep reasoning?** Yes → extended thinking. No → standard response.

This sequence maps to the folder structure in the MWP Starter Kit. The `_router.md` file asks the same kinds of questions to point you (and Claude) to the right place.

5 Quick Reference

Skill	Best for	Layer	Where
Projects	Persistent context for ongoing work	All	Claude.ai
Custom Skills	Repeatable processes and standards	30%	Both
Claude Code	Codebase navigation and development	All	Terminal
MCP Connectors	External tool integration	Infra	Both
Memory	Cross-conversation personalization	Infra	Claude.ai
Code Execution	Data processing and computation	60%	Claude.ai
Artifacts	Interactive tools and prototypes	30%	Claude.ai
File Creation	Professional deliverables (.docx, etc.)	30%	Claude.ai
Web Search	Current information and research	60%/10%	Claude.ai
Extended Thinking	Complex multi-step reasoning	10%	Both

More resources at **clief.notes** and **eduba.io**

New assets drop monthly for Premium and VIP members.